

Documentazione Progetto: LLM Review

1. Architettura del Sistema

Il progetto è stato sviluppato seguendo un'architettura Client-Server, pensata per gestire carichi di lavoro intensivi dovuti all'inferenza di modelli di Intelligenza Artificiale.

- **Back-End:** Realizzato in Python con il framework Flask. È stato scelto Flask per la sua leggerezza e flessibilità nel gestire le rotte web e per la sua perfetta integrazione con script Python complessi, come le chiamate alle API esterne e l'orchestrazione dei modelli locali.
- **Database:** MongoDB (NoSQL). A differenza dei database relazionali tradizionali, MongoDB memorizza i dati in formato BSON, risultando perfetto per gestire la natura flessibile e dinamica dei verdetti restituiti dai vari LLM.
- **Front-End:** Sviluppato con HTML5, CSS3 e JavaScript. Interagisce in modo asincrono con il server per garantire un'esperienza utente fluida senza continui ricaricamenti della pagina.
- **Hardware / Infrastruttura:** Il sistema sfrutta server equipaggiato con tre GPU NVIDIA A800, utilizzate come cluster per l'inferenza ad alte prestazioni dei modelli linguistici locali.

2. Installazione dell'ambiente e messa in funzione

Per configurare e avviare il progetto sul server, la procedura è la seguente:

1. **Python:** Assicurarsi che Python sia installato e aggiunto al PATH del sistema.
2. **MongoDB:** Avere un'istanza locale di MongoDB in esecuzione (porta standard 27017). I dati fisici vengono salvati nella cartella `mongo_data/`.
3. **Ambiente Virtuale (Venv):** Aprire il terminale nella cartella del progetto e attivare l'ambiente virtuale per isolare le librerie:
 - `python -m venv llmrev`
 - Attivazione: `source llmrev/bin/activate`
 - **Dipendenze:** Installare le librerie necessarie (Flask, PyMongo, Pandas, dotenv, openai, tqdm, openpyxl, pypdf, requests, gpt4all, google-generativeai) tramite il comando `pip install -r requirements.txt`.
4. **Configurazione .env:** Creare un file `.env` nella root del progetto per nascondere le variabili sensibili:
 - `FLASK_SECRET_KEY=la_tua_chiave_segreta`
 - `MONGO_URI=mongodb://localhost:27017/`
5. **Avvio Server:** Avviare l'applicativo tramite il comando `python app.py`. La piattaforma sarà raggiungibile all'indirizzo `http://localhost:5000`.

3. Struttura del Progetto e Documentazione del Codice

Il codice è stato suddiviso in moduli per mantenere l'architettura pulita e scalabile.

3.1 File Principali (Back-End)

- **app.py**: È il cuore pulsante dell'applicativo. Gestisce le rotte Flask (login, caricamento PDF, storico, ecc...), smista le richieste verso i motori AI e si occupa del rendering delle pagine HTML.
- **database.py**: Isola la logica di connessione a MongoDB. Contiene tutte le funzioni (query) per inserire paper, verificare il login degli amministratori (`verifica_login`), salvare i verdetti e generare dinamicamente lo storico (`get_storico_analisi`).
- **ai_engine.py**: Gestisce le chiamate alle API Cloud esterne (come GPT-4o-mini e Gemini-2.5-Flash), passando il testo del paper e l'eventuale prompt personalizzato.
- **ai_engine_locale.py**: Modulo fondamentale che orchestra i modelli Open Source (Llama, Mistral, Qwen, Gemma) caricati sulle tre GPU A800 locali, parallelizzando le richieste per abbattere i tempi di inferenza.

3.2 Struttura Front-End (HTML, CSS, JS)

- **templates/**: Contiene le interfacce dinamiche gestite da Jinja2.
 - `base.html`: Lo scheletro del sito (header, navbar, footer).
 - `login.html`: Per gestire il login da parte dell'amministratore.
 - `index.html`: La dashboard per l'upload dei PDF e l'inserimento dei prompt personalizzati.
 - `storico.html`: Tabella dinamica che visualizza i paper analizzati, dotata di un popup modale per consultare i responsi nel dettaglio.
 - `caricamento.html` e `verdetto.html`: Pagine per il feedback visivo durante l'attesa e la visualizzazione finale dei risultati.
- **static/js/**: Presenti tutti i file javascript tra cui `index.js` che gestisce la logica lato client per la home page. Implementa il Drag & Drop dei PDF, la validazione del file e, in particolare, la gestione forzata dell'invio dei dati tramite Fetch API (AJAX) per bypassare le limitazioni di sicurezza dei browser sui form nascosti.

4. Scelte Progettuali

Le decisioni tecniche sono state prese per ottimizzare le prestazioni, l'usabilità e aggirare i colli di bottiglia software.

4.1 Gestione dell'Input tramite Drag & Drop (PDF) Per migliorare l'esperienza utente, si è sostituito il classico tasto "Sfoglia" con un'area Drag & Drop interattiva sviluppata in JavaScript Vanilla. Il sistema valida immediatamente l'estensione del file (solo `.pdf`), fornendo un feedback visivo immediato senza sovraccaricare il server di richieste errate.

4.2 Elaborazione "In-Memory" per aggirare i limiti di Sessione

I modelli LLM restituiscono analisi (motivazioni) estremamente lunghe. Inizialmente, il salvataggio di questi dati nei cookie di sessione di Flask causava crash a causa del limite intrinseco di 4KB. Si è progettato un sistema di parcheggio temporaneo dei dati in RAM tramite i dizionari `RISULTATI_SINGOLI_DB` e `RISULTATI_MASSIVI_DB`. Questo ha azzerato i limiti di peso per il passaggio dei dati dalla fase di elaborazione alla pagina dei verdetti.

4.3 Database NoSQL con relazioni Logiche

La struttura in MongoDB è stata divisa in tre collezioni principali: `utenti_admin`, `dataset_papers` e `verdetti_llm`. Invece di annidare i verdetti direttamente nei paper, si è optato per documenti separati collegati tramite il campo `paper_id`. Questo approccio previene la creazione di documenti giganteschi, facilita l'esportazione in Excel (tramite libreria `pandas`) e rende la generazione dello storico estremamente scalabile.

4.4 Esecuzione Parallela e Threading (AI Locale)

Sfruttando l'infrastruttura di calcolo distribuito (Server Geoscienze + GPU A800), le analisi non avvengono in sequenza, ma in parallelo. Per i task lunghi (come le analisi massive su interi dataset), è stato implementato il modulo `threading` di Python (`elabora_paper_in_background`), che sgancia il calcolo dal ciclo di richiesta HTTP principale, permettendo all'utente di continuare a navigare nel sito senza bloccare il server Flask.